

CS4782 Midterm

Spring 2025

Name: _____

NetID: _____

Email: _____

Write your NetID at the top of every page.

Fill in the NetIDs of your immediate neighbors

I promise to abide by Cornell's Code of Academic Integrity.

Signature: _____

NetID: _____

Epoch 1: True / False Questions

Identify if the below statements are **True** or **False** by answering **T** for true or **F** for false. Please justify your answer with 1-2 sentences if **F (false)**. Correct **T (True)** responses yield 1 point and correct **F (False)** responses yield upto 2 points, 1 for the answer and 1 for the justification.

1. (**T/F**) I submitted the mid-semester course survey form for CS 4/5782 and left constructive and well-meaning feedback.

2. (**T/F**) Imagine a multi-layer perceptron with a single hidden layer. The number of neurons in the hidden layer correspond to the number of linear classifiers/linear decision boundaries that are possible.

Solution:

True.

3. (**T/F**) One of the advantages of training Transformers compared to LSTM models is that Transformers are parallelizable during training.

Solution:

True.

4. (**T/F**) Adding pooling layers decreases the effective receptive field size.

Solution:

False. It increases the effective receptive field size

5. (**T/F**) L1 Regularization produces dense weight vectors with small but non-zero weights, whereas L2 Regularization produces sparse weights with many exactly zero weights.

Solution:

False. It's actually the exact opposite.

6. (**T/F**) If the Adam optimizer runs long enough, it will always converge to the global minimum, independent of initialization and of the loss landscape.

NetID: _____

Solution:

False. There is no guarantee that the local minimum it converge to is also a global minimum (in fact it is extremely unlikely).

7. (**T/F**) GANs are fast during inference time and can generate high quality samples but suffer from mode collapse, whereas diffusion models have high mode coverage and high quality samples but are slower at inference time.



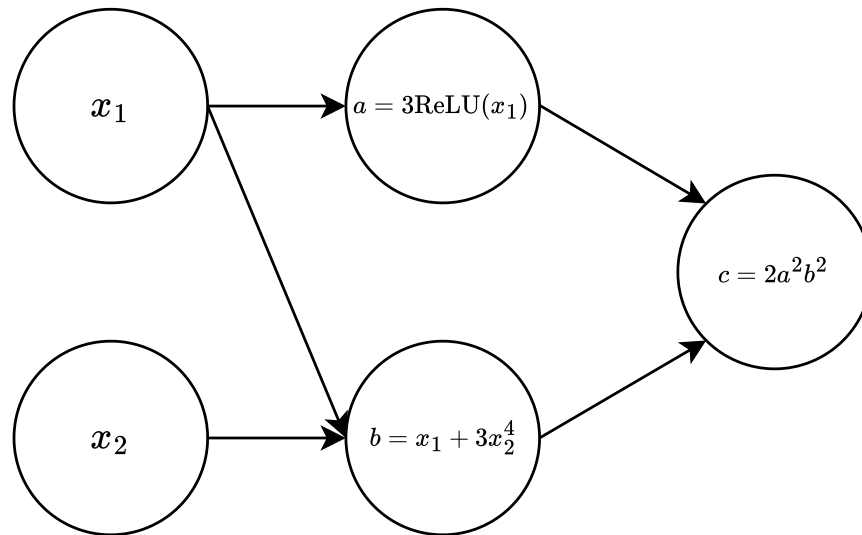
Solution:

True.

NetID: _____

Epoch 2: Backpropagate your way to success! [9 pts]

You are given the following network with inputs x_1 and x_2 and output c :



Calculate the partial derivative of the output c with respect to the inputs x_1 and x_2 . Express your answer in terms of a , b , x_1 , and x_2 . Your solution should include all intermediate derivatives showing how you applied the chain rule to reach the final result. For the sake of this question, you may use ReLU' to denote the derivative of ReLU .

$$\frac{\partial c}{\partial x_1} =$$

$$\frac{\partial c}{\partial x_2} =$$

$$\frac{\partial c}{\partial a} =$$

$$\frac{\partial a}{\partial x_1} =$$

$$\frac{\partial c}{\partial b} =$$

$$\frac{\partial b}{\partial x_1} =$$

$$\frac{\partial b}{\partial x_2} =$$

Solution:

NetID: _____

$$\frac{\partial c}{\partial x_1} = \frac{\partial c}{\partial a} \cdot \frac{\partial a}{\partial x_1} + \frac{\partial c}{\partial b} \cdot \frac{\partial b}{\partial x_1}$$

$$\frac{\partial c}{\partial x_2} = \frac{\partial c}{\partial b} \cdot \frac{\partial b}{\partial x_2}$$

$$\frac{\partial c}{\partial a} = 4ab^2$$

$$\frac{\partial a}{\partial x_1} = 3\text{ReLU}'(x_1)$$

$$\frac{\partial c}{\partial b} = 4a^2b$$

$$\frac{\partial b}{\partial x_1} = 1$$

$$\frac{\partial b}{\partial x_2} = 12x_2^3$$

$$\frac{\partial c}{\partial x_1} = 12ab^2\text{ReLU}'(x_1) + 4a^2b$$

$$\frac{\partial c}{\partial x_2} = 48a^2bx_2^3$$

Epoch 3: Fundamentals Before Fine-Tuning! [5 pts]

1. (2) Identify the correct effects by filling in the appropriate option in the provided blanks:

Increasing the **dropout rate*** is equivalent to a) _____ (increasing/decreasing) model regularization, which may b) _____ (impair/improve) the generalization capability of the model, but if increased too much it may lead to c) _____ (under-fitting/overfitting) and d) _____ (longer/shorter) training times.

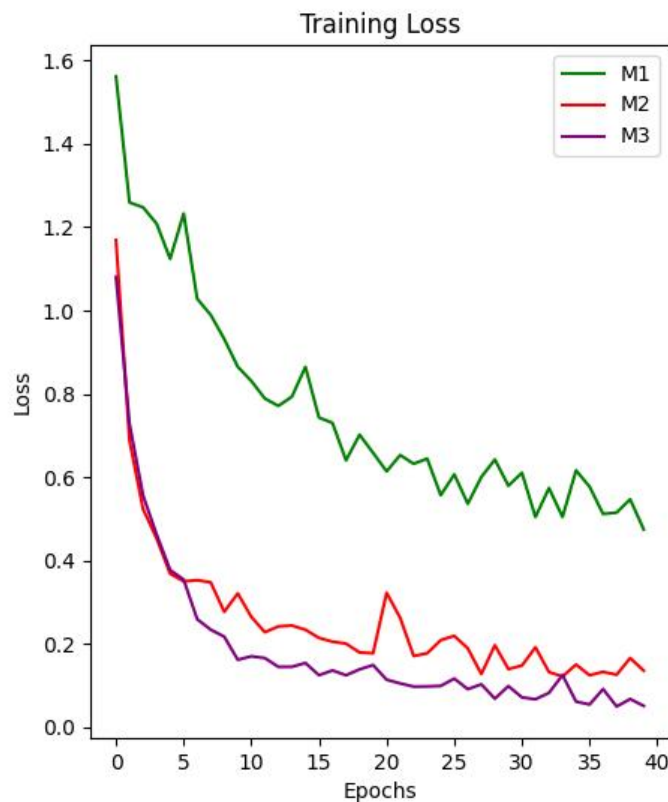
**dropout rate : probability of a neuron being randomly excluded from the network*

Solution:

Increasing the dropout rate is equivalent to a) **increasing** model regularization, which may b) **improve** the generalization capability of the model, but if increased too much it may lead to c) **under-fitting** and d) **longer** training times

2. (3) Consider models M_1 , M_2 and M_3 each trained with a certain amount of dropout. Assume p_1 , p_2 and p_3 represent the **dropout rate*** of M_1 , M_2 and M_3 respectively. **Interpret the order** of the rates based on the **training** loss curves below:

**dropout rate : probability of a neuron being randomly excluded from the network*



$$\left(p_{\square} \right) > \left(p_{\square} \right) > \left(p_{\square} \right)$$

Solution:

NetID: _____

$$(p_1) > (p_2) > (p_3)$$

NetID: _____

Epoch 4: Coding [10 pts]

You have implemented the CNN architecture shown below for an image classification task.

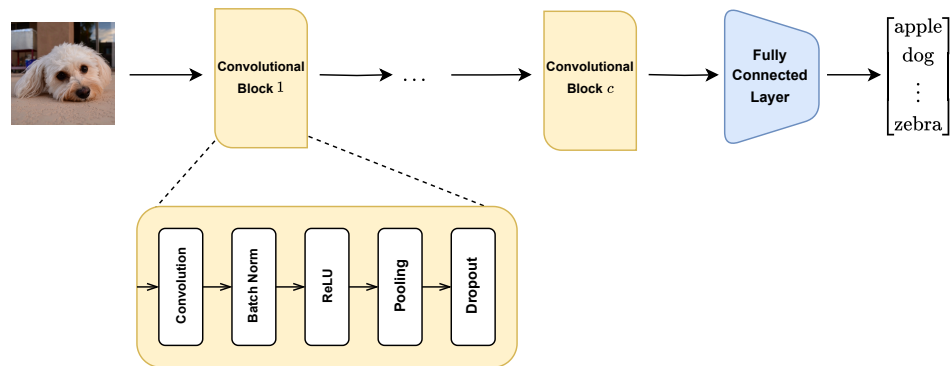


Figure 1: CNN Architecture

Below is your train and inference code:

```
1  def train(model, train_loader, criterion, optim, device='cuda'):
2      running_loss = 0
3      model.train()
4      for images, label in train_loader:
5          images = images.to(device)
6          optim.zero_grad()
7          outputs = model(images)
8          loss = criterion(outputs, label)
9          loss.backward()
10         optim.step()
11         running_loss += loss.item()
12     return running_loss
13
14  def predict(model, test_loader, device='cuda'):
15      model.eval()
16      predictions = []
17      with torch.no_grad():
18         for images, * in test_loader:
19             images = images.to(device)
20             outputs = model(images)
21             predicted = torch.argmax(outputs.data, 1)
22             predictions.extend(predicted.cpu().numpy())
23     return predictions
```

1. (2) How would training be impacted if line 6 (`optim.zero_grad()`) was removed from the code?

2. (2) Why is there an `argmax` operator on line 21?

NetID: _____

3. (2) Which two lines copy the input data to the GPU?

4. (4) Line 15 (`model.eval()`) sets the model into evaluation mode. This is critical if layers behave differently during inference than during training. Which elements in the architecture require the model to be set to eval mode?

Solution:

- (a) The optimizer will apply accumulated gradients from multiple batches, resulting in much larger parameter updates than intended.
- (b) Since the output typically is softmaxed, we want to retrieve the prediction with the highest probability associated with the softmaxed output
- (c) Lines 5 and 19
- (d) BatchNorm (Stops updating running statistics and uses the stored running mean and variance instead of batch statistics) and Dropout (Disables dropout completely (all neurons remain active))

Epoch 5: Teaching computers how to read English [10 pts]

Jack is trying to train an RNN to perform sentence-wise sentiment analysis (i.e. predict if a sentence has positive or negative sentiment.) Below is a figure of his setup:

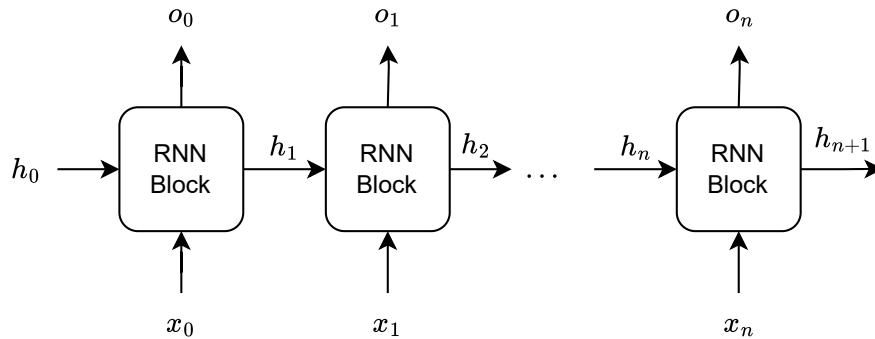


Figure 2: RNN with a single sentence input. x_i represents the $\mathbf{W}$ -dimensional word embedding for word i in a single sentence. h_i represents the $\mathbf{D}$ -dimensional hidden state. o_i represents the $\mathbf{D}$ -dimensional output of an RNN block.

1. **(3)** Assume a sentence is either of positive (+1) or of negative (-1) sentiment. What loss would Jack use for the sentiment analysis problem? (You can name it or write down a formula.)

Solution:

Binary Cross Entropy Loss: $\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)]$

2. **(4)** Using the figure and variables above, explain how to extend the architecture to turn it into a sentiment classifier.

NetID: _____

Solution:

There are a few ways you can turn this into a sentiment classifier. The most common technique is to take the last hidden state h_{n+1} and feed that through a MLP to perform classification. However, you can also take the average of the hidden states and perform sentiment analysis classification on it.

3. **(3)** For very long sentences, Jack notices that the RNN doesn't perform well. Explain why this might be the case.

Solution:

Vanishing gradients: As the sequence length increases, gradients get multiplied many times during backpropagation. With activation functions like tanh or sigmoid, these gradients tend to become extremely small, making it difficult for the model to learn long-range dependencies.

Epoch 6: Probing the Transformer. [11 pts]

1. (2) Explain why positional embeddings are needed for Transformers but not for RNNs.

Solution:

Transformers need positional embeddings because they process all tokens in parallel, treating input as an unordered set without position information, which would make sentences like "dog chases cat" and "cat chases dog" indistinguishable. RNNs, however, naturally capture positional information through their sequential processing, where each token is processed in order and each hidden state depends on the previous state. This fundamental architectural difference explains why position must be explicitly encoded for transformers but comes built-in with RNNs.

2. (4) Instead of specifying the positional embedding (as a combination of sine and cosine functions), another option is to *learn* the positional vectors. This is common practice with Vision Transformers, but not so common for text input. Explain why learned vectors might be problematic in the context of text.

Solution:

Text has variable length, you only see the very far positions extremely rarely - not often enough to get signal to learn. In contrast, images are always divided in the same number of patches, so you get a healthy training signal for all positional vectors.

3. (5) ALiBi (Attention with Linear Biases) [Press et al. 2022] introduces an alternative to positional embeddings. Instead of adding positional vectors, the authors suggest to modify the attention mechanism from

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) \cdot V$$

to

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} + B \right) \cdot V$$

where B is a bias matrix defined as

$$B_{i,j} = m \cdot (i - j),$$

NetID: _____

and the constant m is learned for each layer. Assume you have a GPT type model (decoder only) and i refers to the index of the query and j to the key. Name one advantage and one disadvantage of this method compared to the standard positional embedding.

Solution:

Advantages:

- (a) Much lower memory cost
- (b) Position is computed relative instead of absolute, which can be more efficient.
- (c) It encodes a bias that tokens that are far apart are less relevant to each other

Disadvantages:

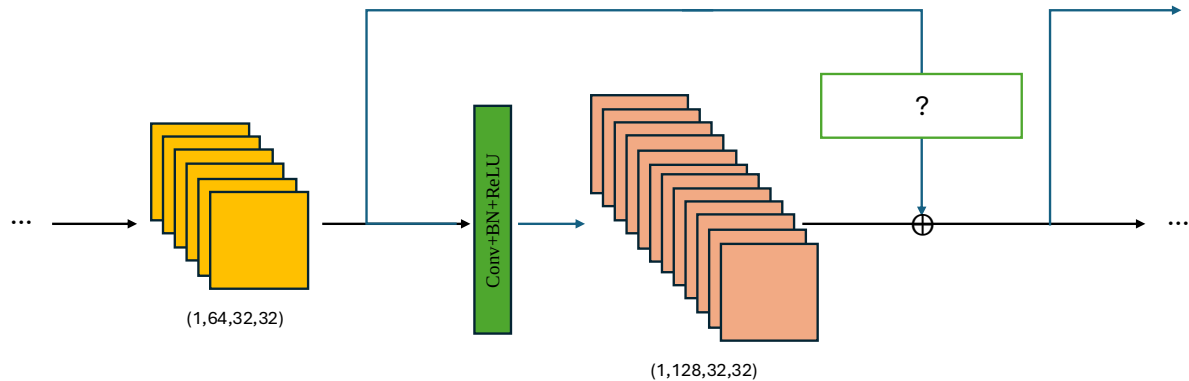
- (a) Not as expressive/flexible
- (b) Might give too little attention to far away tokens

NetID: _____

Epoch 7: Images are all you need [10 pts]

1. ResNets [4 pts]

The basic residual formulation $y = F(x) + x$ requires that $F(x)$ and x have the same dimensions for element-wise addition. When the channel dimensions increase (for example, from 64 to 128 channels), the shapes of $F(x)$ and x become incompatible, making direct addition impossible. What would you do to handle the dimension mismatch in the skip connection? Please explain your reasoning.



Solution:

Add 1×1 convolutions in the skip path to handle the dimension mismatching. This 1×1 convolution projects the input to match the output dimensions, making the addition operation valid while providing learnable parameters to optimize the transformation between different feature spaces.

NetID: _____

2. Self-supervised Learning [6 pts]

1. **(2)** Which model architecture for modeling visual data typically require more training data: Vision Transformers or Convolutional Neural Networks? Please explain your reasoning.

Solution:

1. Inductive biases: CNNs have strong inductive biases through local receptive fields, weight sharing, and translation equivariance built into their architecture. Vision Transformers have weaker inductive biases, relying primarily on self-attention mechanisms that allow any patch to attend to any other patch from the first layer.
 2. Feature hierarchy: CNNs build features hierarchically through stacked convolutional layers that gradually increase receptive field size. Vision Transformers process all patches simultaneously and rely on position embeddings and self-attention to establish spatial relationships.
2. **(4)** To train a vision-language model with the same model architecture, Snehal manually annotated some images and then used them to train her model. Meanwhile, Travis collected a large amount of image-text pairs from the internet and used them to train his model. Explain one significant advantage and one potential limitation of Travis's approach compared to Snehal's.

Solution:

Advantage: Scale and diversity - Travis's internet-sourced image-text pairs expose the model to a vastly broader range of concepts, contexts, and visual styles than Snehal's manually annotated dataset, enabling more generalizable representations and better zero-shot transfer to unseen domains.

Limitation: Travis's internet-sourced data likely contains significant noise, inaccuracies, and societal biases that can become embedded in the model's representations, potentially leading to unfair or harmful predictions.

NetID: _____

Epoch 8: Diffusing to the Finish Line! [20 pts]

1. VAE Loss [4 pts]

As a refresher, the standard VAE loss function is defined as the following:

$$\mathcal{L}_{\text{VAE}}(\theta, \phi) = -\mathbb{E}_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] + D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))$$

Identify and explain each component of the VAE loss function (red and blue). What do the two terms represent and are those terms tractable?

Solution:

1. The term $-\mathbb{E}_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)]$ represents the reconstruction loss. It measures how well the decoder can reconstruct the original input data after encoding it to latent space and sampling from it. This term ensures that the VAE learns to encode meaningful information about the input into the latent representation.
2. The term $D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))$ represents the KL divergence between the encoder's distribution $q_{\phi}(z|x)$ and the prior distribution $p(z)$ (typically a standard normal distribution). This regularization term encourages the encoder to produce latent representations that follow the prior distribution, which enables random sampling for generation.

NetID: _____

2. Forward Diffusion Process [14 pts]

Let us now move on to diffusion models. Consider a discrete-time forward diffusion process $\{x_0, x_1, \dots, x_T\}$ defined as follows:

$$q(x_t \mid x_{t-1}) \sim \mathcal{N}\left(\sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}\right) \quad \text{for } t = 1, 2, \dots, T,$$

where $\beta_t \in (0, 1)$ is a variance schedule that typically increases with t . Let us define $\alpha_t = 1 - \beta_t$ and

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Recall that $x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_{t-1} = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}$, where $\epsilon_{t-1} \sim \mathcal{N}(0, \mathbf{I})$ is the sampled noise for each step. Show that the *marginal* distribution $q(x_t \mid x_0)$ is a Gaussian with mean $\sqrt{\bar{\alpha}_t} x_0$ and covariance $(1 - \bar{\alpha}_t) \mathbf{I}$. Let's do it step by step:

1. **(2)** Let's get x_1 from x_0 by simply adding a single step of noise based on $q(x_t \mid x_{t-1})$.

2. **(3)** Do the same noise-adding step from x_1 to x_2 , and then express x_2 using x_0 , ϵ_0 , and ϵ_1 .

3. **(4)** Rewrite x_2 using only a single noise term $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, you may find this useful: $X, Y \sim \mathcal{N}(0, \mathbf{I})$, $Z = aX + bY \Rightarrow Z \sim \mathcal{N}(0, a^2 + b^2)$.

NetID: _____

4. **(5)** Repeat the previous step recursively until step t . Remember that we define $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. Let's replace all the multiplications of α_t with $\bar{\alpha}_t$. Now you will have a one-step process to add noise!

Solution:

Consider the forward diffusion process defined as:

$$x_t \mid x_{t-1} \sim \mathcal{N}\left(\sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}\right)$$

Let us define:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

We aim to show:

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I}\right)$$

We start from the recursive definition:

$$x_1 = \sqrt{\alpha_1} x_0 + \sqrt{1 - \alpha_1} \epsilon_0 \tag{1}$$

$$x_2 = \sqrt{\alpha_2} x_1 + \sqrt{1 - \alpha_2} \epsilon_1 \tag{2}$$

$$= \sqrt{\alpha_2} \left(\sqrt{\alpha_1} x_0 + \sqrt{1 - \alpha_1} \epsilon_0 \right) + \sqrt{1 - \alpha_2} \epsilon_1 \tag{3}$$

$$= \sqrt{\alpha_1 \alpha_2} x_0 + \sqrt{\alpha_2 (1 - \alpha_1)} \epsilon_0 + \sqrt{1 - \alpha_2} \epsilon_1 \tag{4}$$

$$= \sqrt{\alpha_1 \alpha_2} x_0 + \sqrt{1 - \alpha_2 \alpha_1} \epsilon \tag{5}$$

Continuing this recursively, we get:

$$x_t = \sqrt{\prod_{i=1}^t \alpha_i} x_0 + \sqrt{1 - \prod_{i=1}^t \alpha_i} \epsilon_0$$

Since the $\epsilon_i \sim \mathcal{N}(0, \mathbf{I})$ are independent and Gaussian, the sum of these scaled noises is still Gaussian. The variance becomes:

$$\text{Var}(x_t \mid x_0) = (1 - \bar{\alpha}_t) \mathbf{I}$$

Hence, we conclude:

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I}\right)$$

Solution:

When t becomes large, we get: $q(x_t \mid x_0) \approx \mathcal{N}(0, \mathbf{I})$.

NetID: _____

3. Diffusion Backwards Pass [2 pts]

Let's examine the backward pass (denoising) in diffusion models. As discussed in class, we want to obtain $q(x_{t-1} | x_t)$ that can both help us denoise the image and serve as ground truth to train our network. Using Bayes' rule, this can be rewritten as:

$$q(x_{t-1} | x_t) = \frac{q(x_t | x_{t-1}) q(x_{t-1})}{q(x_t)}.$$

We have $q(x_t | x_{t-1})$ by definition, but unfortunately, $q(x_{t-1})$ and $q(x_t)$ are not tractable. What trick is used in diffusion models to make this denoising distribution tractable during training? You are allowed to directly use the distribution $q(\cdot)$ or conditional distribution $q(\cdot | \cdot)$ in the answer.

Solution:

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1}, x_0) q(x_{t-1} | x_0)}{q(x_t | x_0)}.$$

NetID: _____

Epoch 9: Your model has converged! (Optional)

Congratulations on finishing the exam! We hope you found it both educational and manageable. How did you think the exam went? We would love to hear your thoughts / anything else that's on your mind! 😊